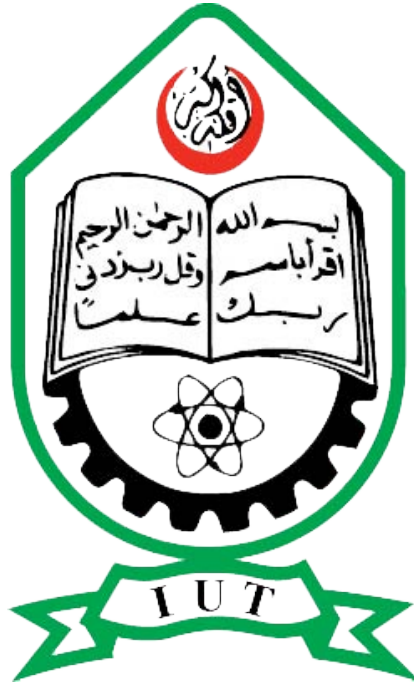




ISLAMIC UNIVERSITY OF TECHNOLOGY

MACHINE LEARNING LAB

CSE 4622

Lab 2

Author:

Tasnif Emran

220041135

Contents

1	Task 1: Dataset Overview and Basic Statistics	2
2	Task 2: Data Cleaning and Imputation	3
3	Task 3: Encoding and Scaling	4
4	Task 4: Data Visualization and Analysis	5
4.1	Task 4(a): Fare Distribution Before and After Scaling	5
4.2	Task 4(b): Survival Counts	6
4.3	Task 4(c): Fare Distribution by Passenger Class	8
4.4	Task 4(d): Pearson Correlation Heatmap	10
4.5	Task 4(e): Age Distribution by Sex (KDE Plot)	12
5	Task 5: Survival Rate Analysis — Women and Children First	13

1 Task 1: Dataset Overview and Basic Statistics

Code 1: Data loading and preprocessing code for Titanic dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.preprocessing import LabelEncoder,
    OneHotEncoder
from sklearn.preprocessing import MinMaxScaler,
    StandardScaler

file_path = "/content/titanic.csv"
df = pd.read_csv(file_path)

df.head(10)

print(f"Shape: {df.shape}")
print(f"\nColumns: {df.columns.tolist()}")
print(f"\nDataTypes: {df.dtypes}")

num_cols = df.select_dtypes(include=['number']).columns.
    tolist()
print(f"Numerical Columns: {num_cols}")

cat_cols = df.select_dtypes(include=['object']).columns.
    tolist()
print(f"Categorical Columns: {cat_cols}")

average = df["Fare"].mean()
print(f"Mean fare: {average}")

higher_average_salary = df[df['Fare'] > average]
print("\n=====")
print(higher_average_salary['Name'].values)
print("\n=====")
```

```
missing_percentages = (df.isnull().sum() / len(df) *
                        100).sort_values(ascending=False)
print(missing_percentages)
```

The dataset used in this lab is the well-known Titanic passenger dataset. It contains records of 891 passengers across 12 features: `PassengerId`, `Survived`, `Pclass`, `Name`, `Sex`, `Age`, `SibSp`, `Parch`, `Ticket`, `Fare`, `Cabin`, and `Embarked`.

Out of these, the numerical columns are `PassengerId`, `Survived`, `Pclass`, `Age`, `SibSp`, `Parch`, and `Fare`. The categorical columns are `Name`, `Sex`, `Ticket`, `Cabin`, and `Embarked`.

The mean ticket fare across all passengers is approximately **£32.20**, while the median is only **£14.45**. The large gap between mean and median already hints at a heavily right-skewed fare distribution — a small group of passengers paid very high fares, which pulls the mean upward. Only 211 out of 891 passengers paid above the mean fare, meaning the majority of passengers were travelling on relatively cheap ticket.

Regarding missing data:

- `Cabin`: 77.1% missing — almost unusable as-is.
- `Age`: 19.9% missing — significant, needs imputation.
- `Embarked`: 0.22% missing — only 2 records, easy to fix.

The `Cabin` column was dropped due to the extremely high proportion of missing values. For `Age`, the median age (28 years) was used for imputation since it is more robust to the skewed distribution than the mean. For `Embarked`, the mode (port ‘S’ — Southampton) was used to fill the two missing records.

2 Task 2: Data Cleaning and Imputation

Code 2: Handling missing values in Titanic dataset

```
df.drop(columns=['Cabin'])

age_median = df['Age'].median()
df['Age'] = df['Age'].fillna(age_median)
```

```

print(f"Missing in Age after imputation: {df['Age'].
      isnull().sum()}")

embarked_mode = df['Embarked'].mode()[0]
print(f"Most common port: {embarked_mode}")

df['Embarked'] = df['Embarked'].fillna(embarked_mode)

print(f"Missing in Embarked after imputation: {df['
      Embarked'].isnull().sum()}")

```

After dropping Cabin and imputing Age and Embarked, the dataset has no remaining missing values.

3 Task 3: Encoding and Scaling

Code 3: Encoding categorical variables and scaling features

```

le = LabelEncoder()
df['Sex_encoded'] = le.fit_transform(df['Sex'])

print("Classes:", le.classes_)
print(df[['Sex', 'Sex_encoded']].head())

embarked_dummies = pd.get_dummies(df['Embarked'], prefix
    = 'Embarked', drop_first=True)
print(embarked_dummies.head())

minmax = MinMaxScaler()
df[['Age_minmax', 'Fare_minmax']] = minmax.fit_transform
    (df[['Age', 'Fare']])

print(df[['Age', 'Age_minmax', 'Fare', 'Fare_minmax']].
    head())

```

Label Encoding for Sex: The Sex column was encoded using LabelEncoder: female → 0, male → 1. This works fine here since sex is binary.

One-Hot Encoding for Embarked: The Embarked column (values: C, Q, S) was transformed using `pd.get_dummies` with `drop_first=True`,

producing two new boolean columns removing the redundancy of the 3rd column which is unnecessary.

Min-Max Scaling: Age and Fare were normalized to the $[0, 1]$ range using `MinMaxScaler`.

4 Task 4: Data Visualization and Analysis

4.1 Task 4(a): Fare Distribution Before and After Scaling

Code 4: Histogram of fare distribution before and after scaling

```
df_raw = pd.read_csv(file_path)

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

axes[0].hist(df_raw['Fare'], bins=30, color='salmon',
             edgecolor='white')
axes[0].set_title("Fare Distribution before")

axes[1].hist(df['Fare'], bins=30, color='steelblue',
             edgecolor='white')
axes[1].set_title("Fare Distribution after")

plt.show()
```

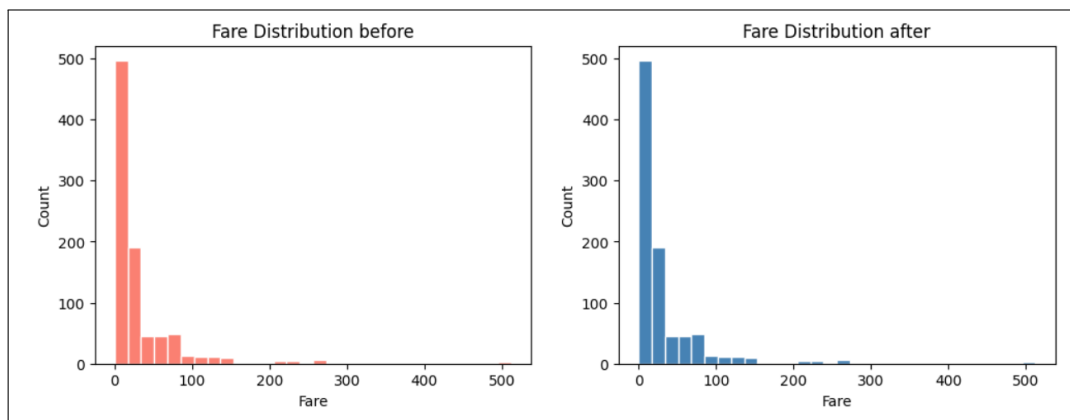


Figure 1: Figure 1: Fare distribution before (left) and after Min-Max scaling (right)

What the graphs show is a very strong right skew in the fare data. The vast majority of passengers (roughly 600+) paid very low fares (under £50, or under 0.1 normalized), while a tiny fraction paid extremely high amounts — the maximum being £512.33. This skew makes sense: the Titanic carried many Third Class passengers (491 out of 891) who paid low fares, compared to 216 First Class passengers who paid significantly more.

The reason the distribution looks almost like an exponential decay is that ticket prices were not uniformly distributed — they were heavily stratified by class, and most passengers were in the lower class brackets.

4.2 Task 4(b): Survival Counts

Code 5: Bar chart of survival counts

```
survival_counts = df['Survived'].value_counts().
    sort_index()

plt.figure(figsize=(6, 4))
plt.bar(['Not Survived', 'Survived'], survival_counts.
    values,
        color=['salmon', 'steelblue'], edgecolor='white'
    )
```

```
plt.title("Total Passengers: Survived vs Not Survived")
plt.xlabel("Survival Status")
plt.ylabel("Number of Passengers")

plt.show()
```

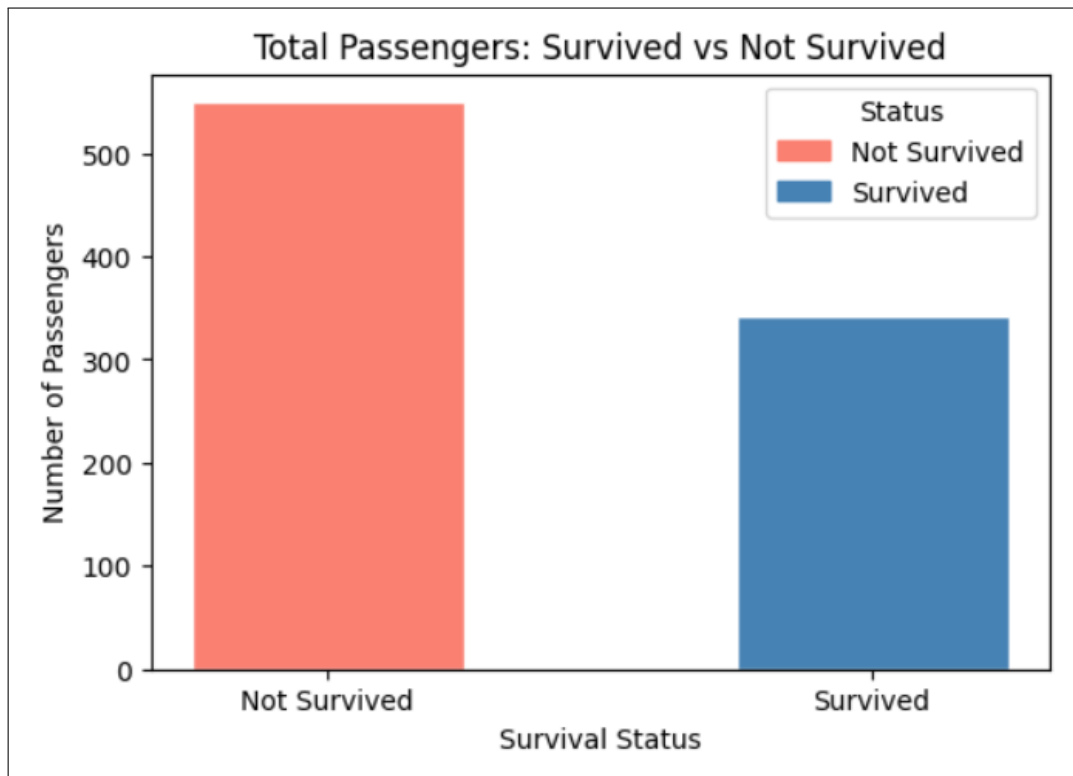


Figure 2: Total count of passengers who survived vs did not survive.

Figure 2 shows a clear imbalance: 549 passengers did not survive while only 342 did, giving an overall survival rate of just **38.38%**. This is an important observation because the classes are imbalanced.

The reason more people died than survived comes down to the timeline of the disaster itself. The ship sank rapidly after hitting the iceberg, and there were not enough lifeboats for everyone. Priority was given to women, children, and First Class passengers.

4.3 Task 4(c): Fare Distribution by Passenger Class

Code 6: Box plot of fare by passenger class

```
plt.figure(figsize=(8, 5))

sns.boxplot(
    data=df,
    x='Pclass',
    y='Fare',
    palette={1: 'steelblue', 2: 'darkorange', 3: 'seagreen'}
)

plt.title("Fare Distribution by Passenger Class")
plt.xlabel("Passenger Class")
plt.ylabel("Fare ( )")

plt.show()

median_fare_by_class = df.groupby('Pclass')['Fare'].
    median()
print(median_fare_by_class)
```

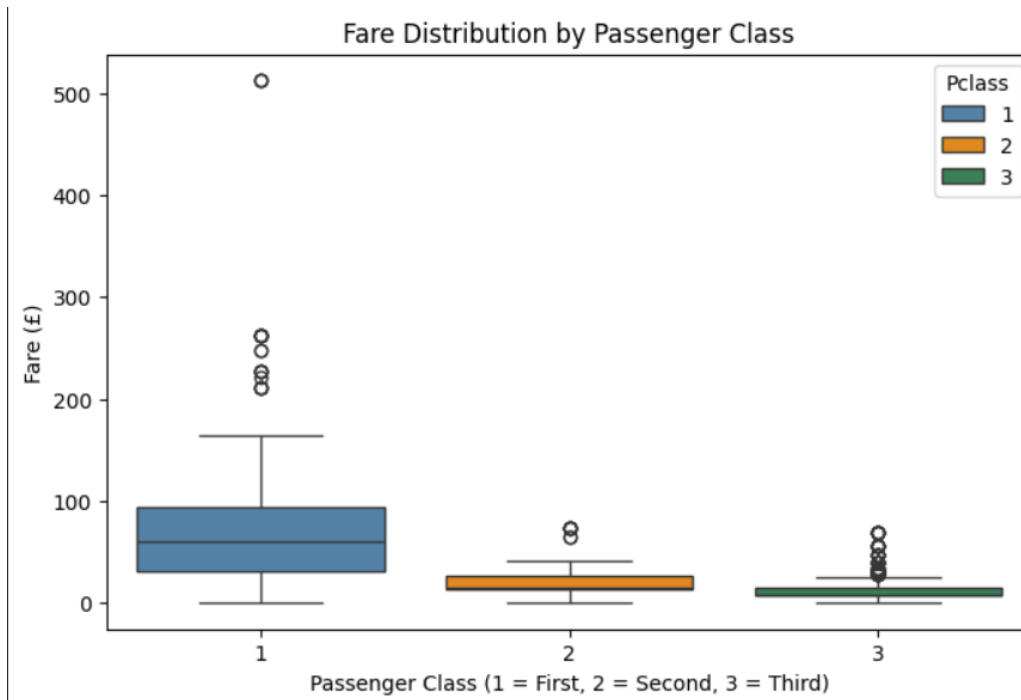


Figure 3: Box plot of fare distribution grouped by passenger class.

Figure 3 shows the box plots of fare for each class. A few things stand out immediately:

Class 1 (steelblue): The median fare is £60.29, with an IQR ranging from £30.92 to £93.50. The box itself is quite wide, meaning fares within First Class were quite variable — some passengers paid around £30 while others paid several hundred pounds. There are 20 outliers, mostly high-paying passengers who booked premium suites.

Class 2 (orange): The median is £14.25, with a very tight IQR (£13.00 to £26.00). The box is narrow and compact, indicating Second Class fares were fairly consistent.

Class 3 (green): The median is just £8.05, the lowest of all. Despite having the tightest absolute spread (IQR: £7.75 to £15.50), Class 3 has the most outliers — 52 in total. This is somewhat counterintuitive at first: why would the cheapest class have so many outliers? The answer is that Class 3 also had by far the most passengers (491), so even a small percentage of passengers with above-normal fares creates a large number of data points that fall outside the whiskers.

The overall trend — fares decreasing from Class 1 to Class 3 — is obvious, but the box plot also reveals that First Class was far more heterogeneous in pricing than the other two classes.

4.4 Task 4(d): Pearson Correlation Heatmap

Code 7: Correlation heatmap of numerical features

```
numeric_df = df.select_dtypes(include=[np.number])
corr_matrix = numeric_df.corr()

plt.figure(figsize=(10, 7))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm',
            center=0)

plt.title("Pearson Correlation Heatmap")
plt.show()

survived_corr = corr_matrix['Survived'].drop('Survived')
                .abs().sort_values(ascending=False)
print(survived_corr.head(3))
```

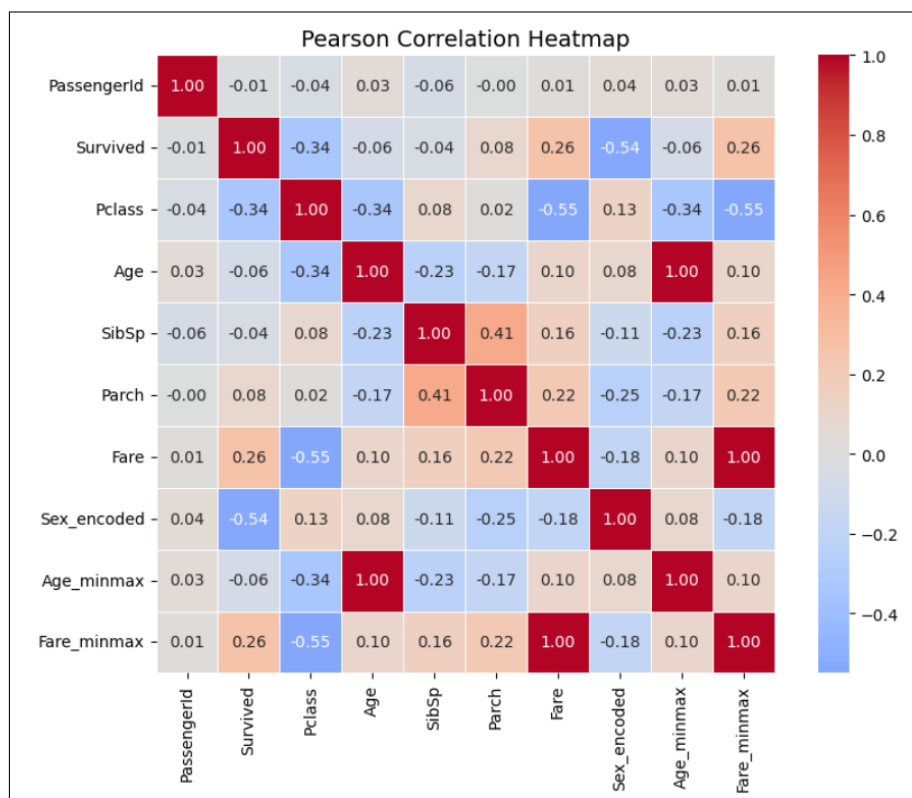


Figure 4: Pearson correlation heatmap of all numerical features.

Figure 4 shows the pairwise Pearson correlations between all numerical features. The top 3 features most correlated (in absolute value) with **Survived** are:

1. **Sex_encoded** ($r = -0.54$): The strongest predictor of survival. The negative sign means males (encoded as 1) had lower survival rates than females (encoded as 0). This is the largest correlation by a significant margin.
2. **Pclass** ($r = -0.34$): Higher class number (i.e., lower socioeconomic status) is associated with lower survival. Again negative: Class 3 passengers survived at far lower rates than Class 1.
3. **Fare** ($r = +0.26$): Higher fare is weakly positively correlated with survival. This is partly a proxy for class — passengers who paid more

were in higher classes, and higher classes survived more. *But even within a class, a higher fare might reflect a better cabin location (closer to lifeboats).*

Other notable correlations in the heatmap:

- **Fare** and **Pclass** ($r = -0.55$): Strong negative correlation, as expected — lower class means lower fare.
- **SibSp** and **Parch** ($r = +0.41$): Moderate positive correlation — passengers with siblings/spouses on board also tended to have parents/children on board, i.e., family travel patterns.
- **Age** and **Pclass** ($r = -0.37$): Older passengers were slightly more likely to be in higher (lower-numbered) classes.

4.5 Task 4(e): Age Distribution by Sex (KDE Plot)

Code 8: KDE plot of age distribution by sex

```
df_plot = df.copy()
df_plot['Sex'] = df_raw['Sex']

plt.figure(figsize=(8, 4))

sns.kdeplot(data=df_plot[df_plot['Sex']=='female'], x='Age',
            fill=True, color='salmon', label='Female')

sns.kdeplot(data=df_plot[df_plot['Sex']=='male'], x='Age',
            fill=True, color='steelblue', label='Male')

plt.title("Age Distribution by Sex (KDE)")
plt.legend()
plt.show()
```

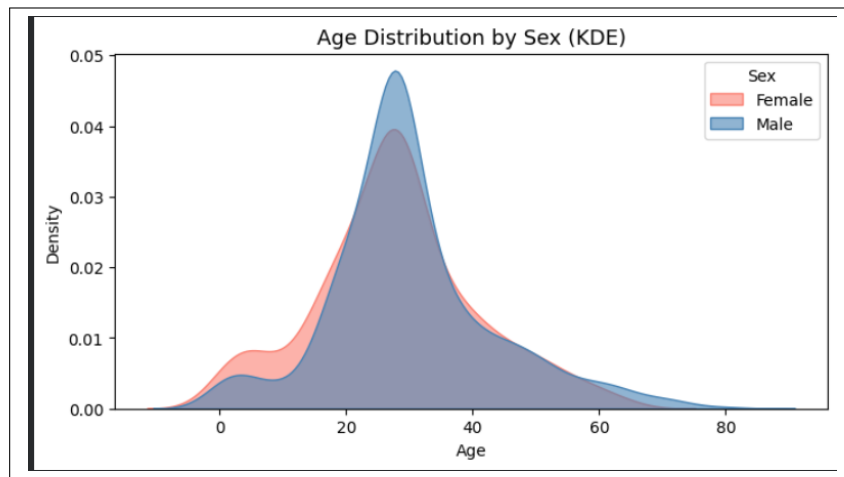


Figure 5: Kernel Density Estimate of age distribution, split by sex.

Figure 5 shows the KDE of ages for males and females separately.

Both distributions peak around age 20–30 and are broadly similar in shape — which makes sense since this is a general passenger population. The female distribution (salmon) has a slightly shorter and wider shape overall, which could partly be a result of the smaller female sample size (314 vs. 577 males). Males have a slightly higher mean age (30.1 years vs. 27.9 years for females), but the difference is not dramatic.

The overlapping nature of the two curves tells us that sex alone is not a strong predictor of age on the Titanic.

5 Task 5: Survival Rate Analysis — Women and Children First

Code 9: Survival analysis by sex and age group

```
df_plot = df.copy()
df_plot['Sex'] = df_raw['Sex']

df_plot['AgeGroup'] = pd.cut(
    df_plot['Age'],
    bins=[0, 12, 18, 60, 100],
    labels=['Child', 'Teen', 'Adult', 'Senior'])
```

```

)

fig, axes = plt.subplots(1, 2, figsize=(13, 5))

sns.barplot(data=df_plot, x='Sex', y='Survived',
            estimator='mean', ax=axes[0])
axes[0].set_title("Survival Rate by Sex")

sns.barplot(data=df_plot, x='AgeGroup', y='Survived',
            estimator='mean', ax=axes[1])
axes[1].set_title("Survival Rate by Age Group")

plt.tight_layout()
plt.show()

```

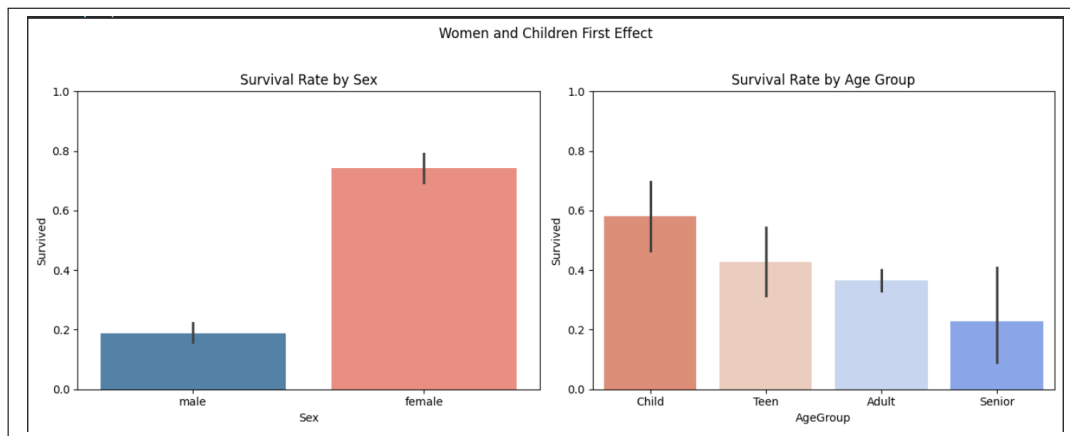


Figure 6: Survival rate by sex (left) and by age group (right).

Left plot — Survival Rate by Sex: Female passengers had a survival rate of approximately **74.2%**, while male passengers survived at only **18.9%**. The difference is enormous — nearly a 4:1 ratio. This is not a coincidence or a statistical artifact; it directly reflects the deliberate boarding order enforced by the Titanic's crew. Women were given priority access to lifeboats. The bar chart makes this stark contrast visually undeniable.

Right plot — Survival Rate by Age Group: The survival rates are:

- Children (0–12): **57.97%**

- Teens (13–18): **42.86%**
- Adults (19–60): **36.58%**
- Seniors (60+): **22.73%**

Children had the highest survival rate, consistent with the “children first” policy. The survival rate decreases monotonically with age group, which is an interesting finding. Seniors had the worst survival rate, possibly because they were slower to move to the lifeboats, or because older passengers in Third Class had less physical ability to navigate the ship during the chaos.

The teen survival rate (42.86%) is between children and adults, which makes sense — teens were partially prioritised but not as strictly as young children.

In conclusion, the famous ‘women and children first’ policy does reflect in the Titanic survival data.